

SÜLEYMAN DEMİREL ÜNİVERSİTESİ



BİL-304 BİLGİSAYAR MİMARİSİ Dr. Öğr. Üyesi Fatih GÖKÇE

11. Hafta

Bölüm / Hafta Hedefleri

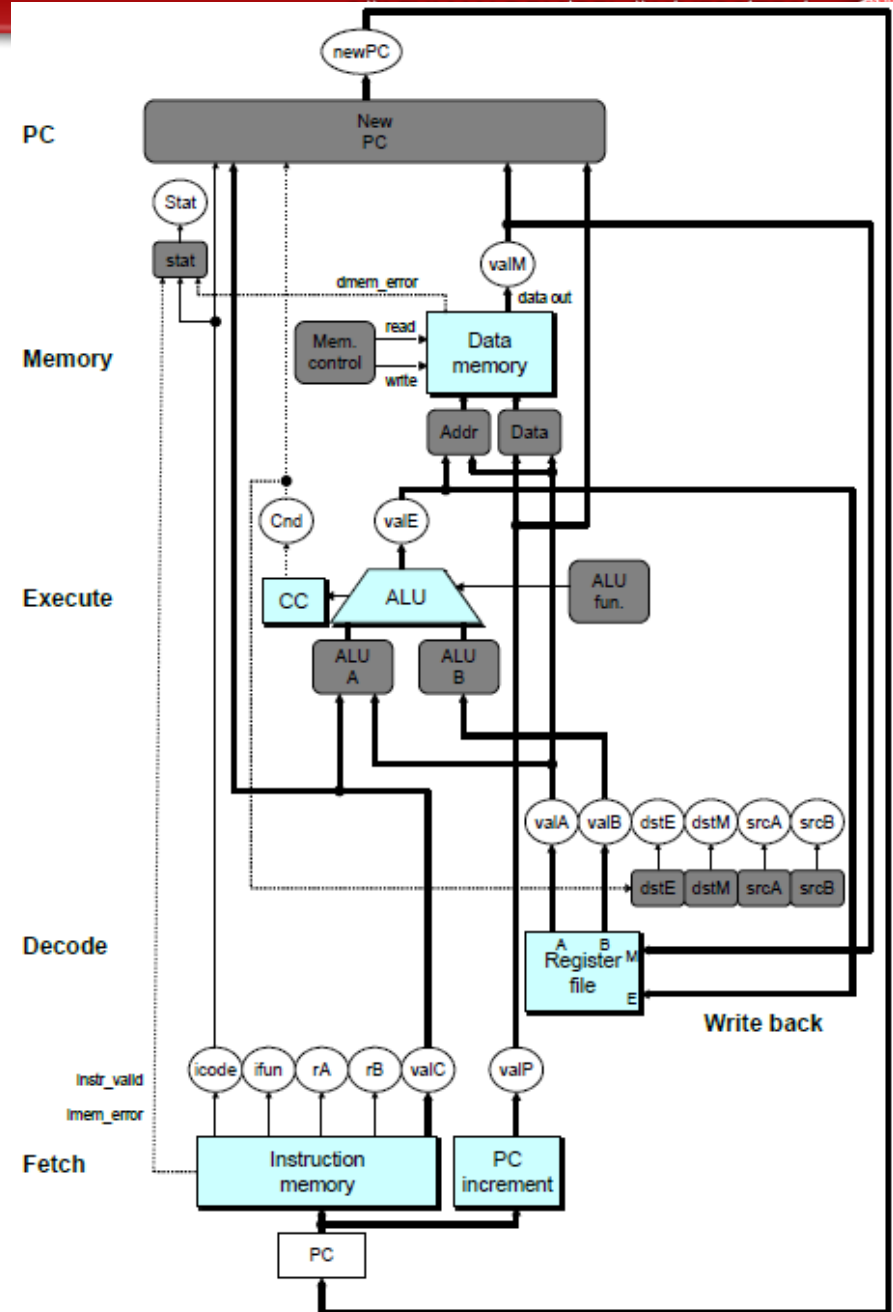
- **Ardışıl İşlemci (SEQ) mimarisinin 2. Kısımını anlatacağımız bu derste**
 - Ardışıl işlemcinin donanımsal yapısının detaylarının,
 - HCL dilinde tanımının öğrenilmesi hedeflenmektedir.

Bu ders notundaki slaytlar ders kitabına ait sunum dosyalarından adapte edilmiştir:
<http://csapp.cs.cmu.edu/>

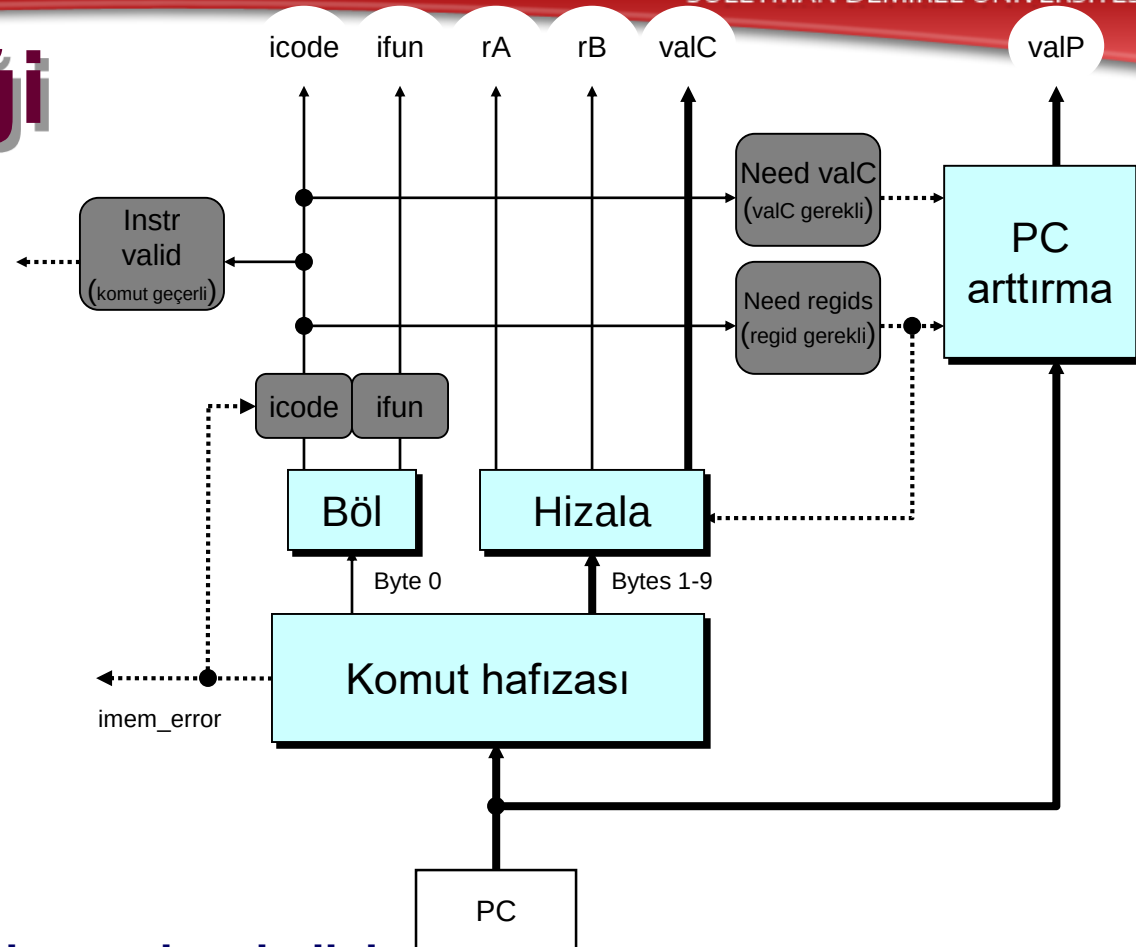
Adapted from slides of the textbook: <http://csapp.cs.cmu.edu/>

SEQ Donanımı

- Mavi kutular: öntasarımlı donanım blokları
 - Örnek: hafızalar, ALU
- Gri kutular: kontrol lojiği
 - HCL'de tanımlanır
- Beyaz oval kutular: sinyal etiketleri
- Kalın hatlar: 64-bit sözcük değerleri
- İnce hatlar: 4-8 bit değerler
- Kesikli noktalı hatlar: 1-bit değerler
- Sağdaki şekildeki İngilizce terimler ilerleyen slaytlarda her bir kısım ayrı ayrı detaylandırılırken çevrilecektir.



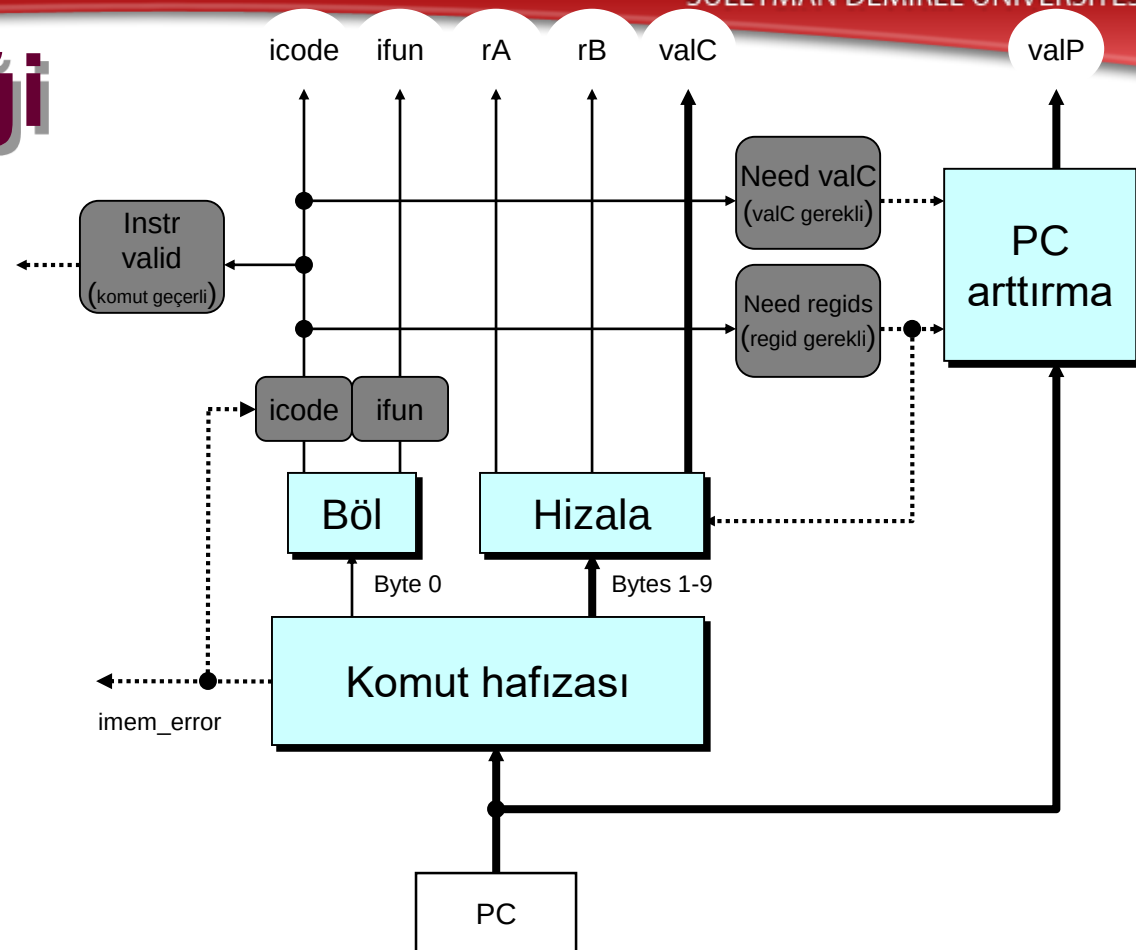
Fetch Lojiği



Öntasarımlı bloklar

- **PC:** PC değerini içeren kaydedici
- **Komut hafızası:** 10 byte okunması durumunda PC den PC+9'a kadar adreslerdeki byte'lar okunur
 - Geçersiz adres durumunda geçersiz adres hata sinyalini üretir
- **Böl:** Komut byte'ını icode ve ifun kısımlarına ayırır
- **Hizala:** rA, rB ve valC değerlerini çeker

Fetch Lojiği



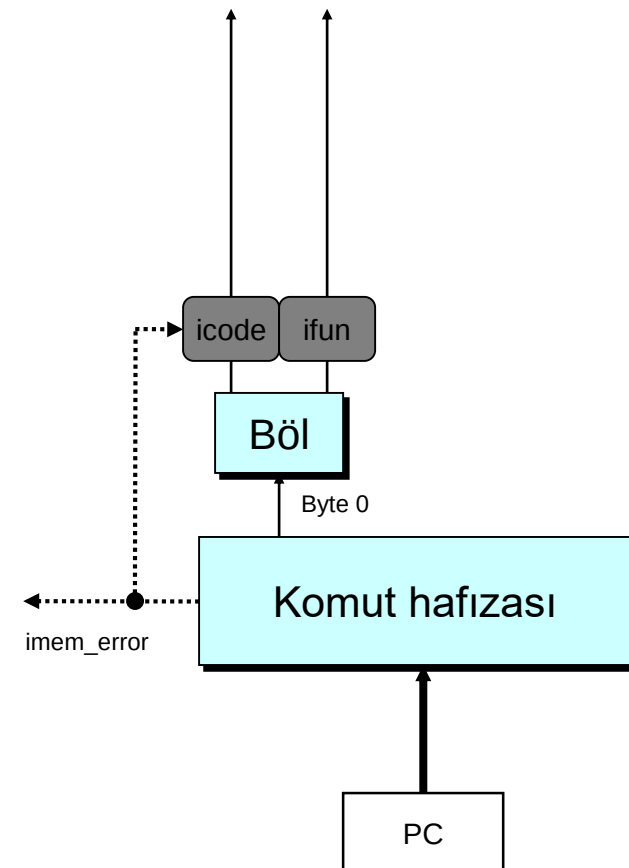
Kontrol lojiği

- **Instr. Valid (komut geçerli):** Bu komut geçerli mi?
- **icode, ifun:** Geçersiz adres durumunda nop komutu üretilir
- **Need regids (regids gerekli):** Komutta kaydedici kullanılıyor mu?
- **Need valC (valC gerekli):** Komutta sabit bir sözcük var mı?

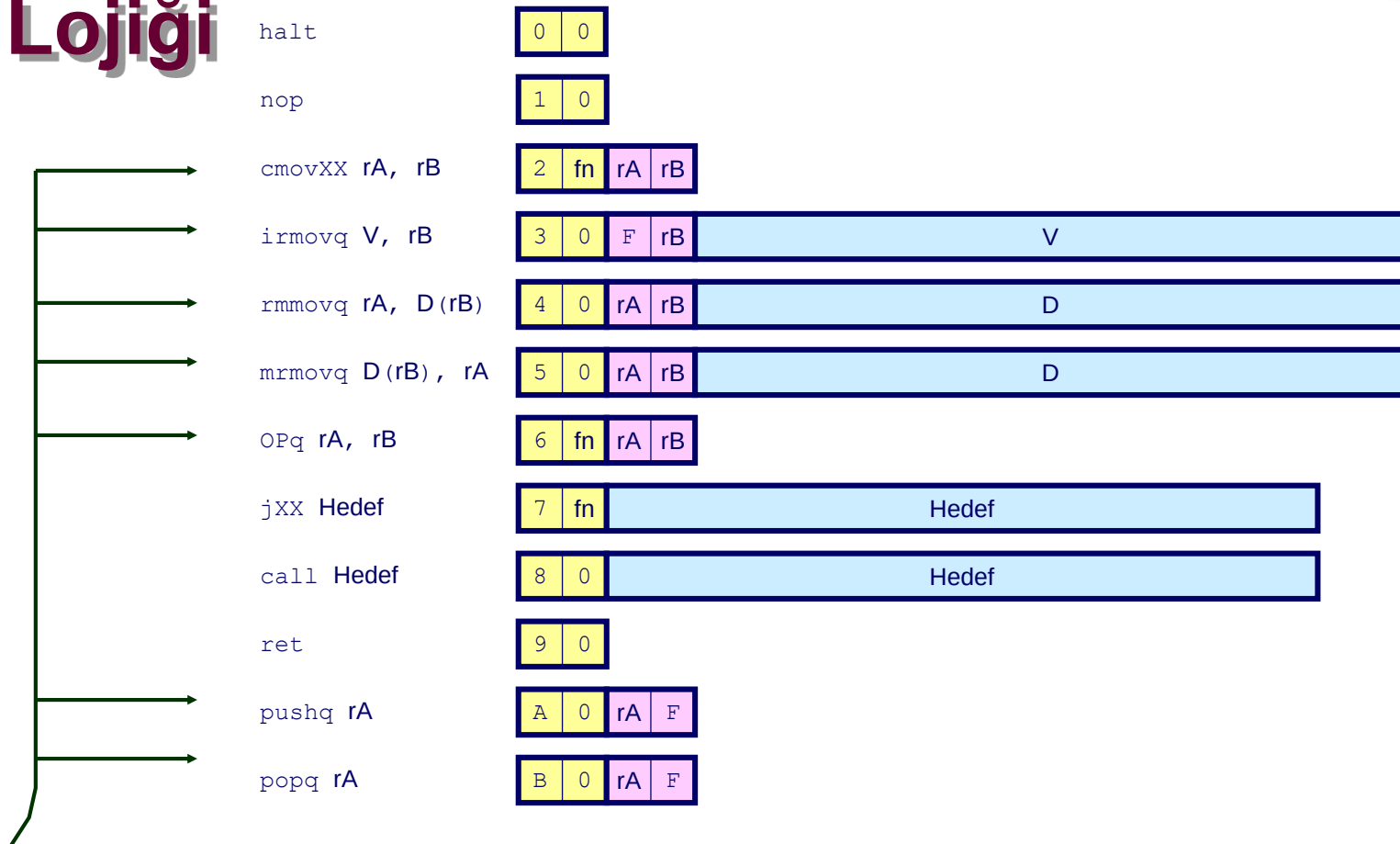
HCL dilinde Fetch Kontrol Lojiği

```
# Komut kodunu belirle
int icode = [
    imem_error: INOP;
    1: imem_icode;
];

# Komut fonksiyonunu belirle
int ifun = [
    imem_error: FNONE;
    1: imem_ifun;
];
```



HCL dilinde Fetch Kontrol Lojigi



```

bool need_regids =
    icode in { IRRMOVQ, IOPQ, IPUSHQ, IPOPOPQ,
               IIRMOVQ, IRMMOVQ, IMRMOVQ };
bool instr_valid = icode in
    { INOP, IHALT, IRRMOVQ, IIRMOVQ, IRMMOVQ, IMRMOVQ,
      IOPQ, IJXX, ICALL, IRET, IPUSHQ, IPOPOPQ };
  
```

Decode Lojîği

Kaydedici Dizisi

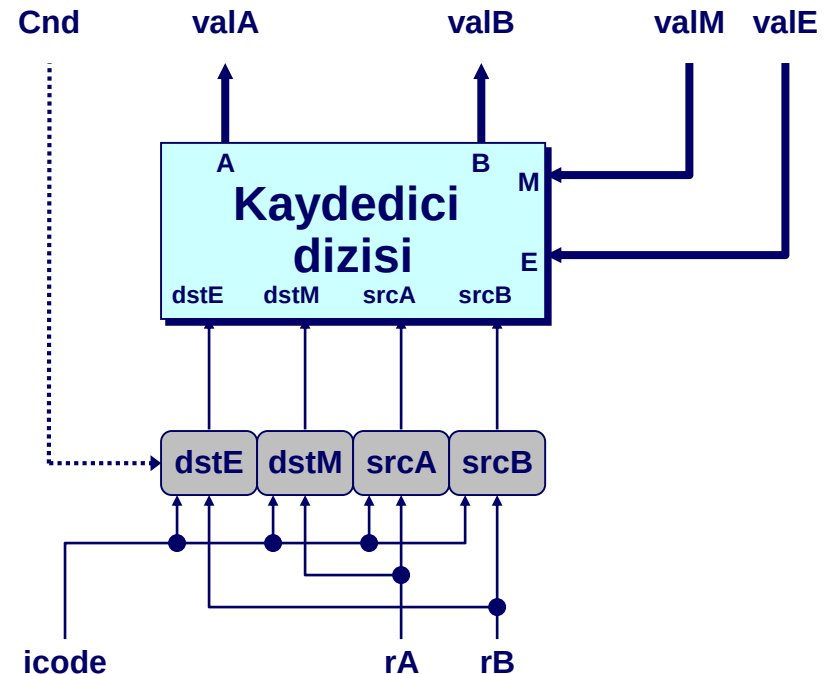
- Okuma portları: A, B
- Yazma portları: E, M
- Adresler kaydedici ID'leri veya 15 (0xF)'tir (kaydedici erişimi olmaması durumunda)

Kontrol Lojîği

- srcA, srcB: Okuma port adresleri
- dstE, dstM: Yazma port adresleri

Sinyaller

- Cnd: Şartlı taşıma yapılp yapılmayacağını belirtir
 - Execute aşamasında hesaplanır



A Değerinin Kaynakları

	OPq rA, rB	
Decode	valA $\leftarrow$ R[rA]	
	cmovXX rA, rB	
Decode	valA $\leftarrow$ R[rA]	
	rmmovq rA, D(rB)	
Decode	valA $\leftarrow$ R[rA]	
	popq rA	
Decode	valA $\leftarrow$ R[%rsp]	
	jXX Dest	
Decode		
	call Dest	
Decode		
	ret	
Decode	valA $\leftarrow$ R[%rsp]	

İşlem yapılacak A değerini oku

İşlem yapılacak A değerini oku

İşlem yapılacak A değerini oku

Yığın pointer'ını oku

İşlem yapılacak bir şey yok

İşlem yapılacak bir şey yok

Yığın pointer'ını oku

```
int srcA = [
    icode in { IRRMOVQ, IRMMOVQ, IOPQ, IPUSHQ } : rA;
    icode in { IPOPOPQ, IRET } : RRSR;
    1 : RNONE; # Kaydediciye gerek yok
];
```

E Değerinin Yazılacağı Yerler

	OPq rA, rB
Write-back	R[rB] ← valE

Sonucu geri yaz

	cmovXX rA, rB
Write-back	R[rB] ← valE

Şarta bağlı olarak
sonucu geri yaz

	rmmovq rA, D(rB)
Write-back	

Hiçbir şey yok

	popq rA
Write-back	R[%rsp] ← valE

Yığın pointer'ını güncelle

	jXX Dest
Write-back	

Hiçbir şey yok

	call Dest
Write-back	R[%rsp] ← valE

Yığın pointer'ını güncelle

	ret
Write-back	R[%rsp] ← valE

Yığın pointer'ını güncelle

```
int dstE = [
    icode in { IRRMOVQ } && Cnd : rB;
    icode in { IIRMOVQ, IOPQ } : rB;
    icode in { IPUSHQ, IPOPOPQ, ICALL, IRET } : RRSP;
    1 : RNONE; # Herhangi bir kaydediciye yazma
```

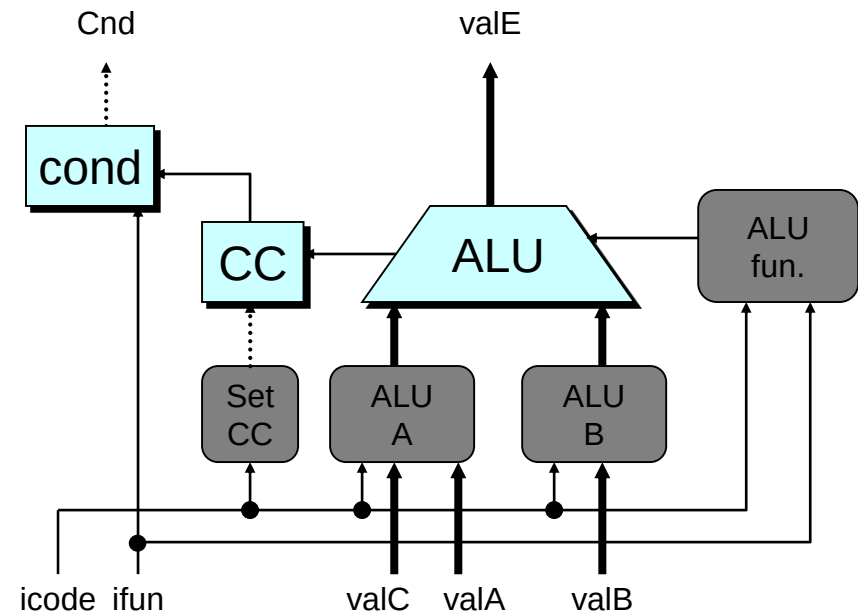
Execute Lojiği

Birimler

- **ALU**
 - İhtiyaç duyulan 4 fonksiyonu gerçekleştirir
 - Durum kodu değerlerini üretir
- **CC**
 - 3 bitlik durum kodlarını içeren kaydedici
- **cond**
 - Şartlı dallanma/taşımaya bayrağını (Cnd) hesaplar

Kontrol Lojiği

- **Set CC:** Durum kodu kaydedicisi yüklenmeli mi?
- **ALU A:** ALU'nun A girişi
- **ALU B:** ALU'nun B girişi
- **ALU fun:** ALU hangi fonksiyonu hesaplamalı?



ALU A

Girişi

	OPq rA, rB	
Execute	$valE \leftarrow valB \text{ OP } valA$	ALU işlemini gerçekleştir
	cmovXX rA, rB	
Execute	$valE \leftarrow 0 + valA$	valA'yı ALU üzerinden aktar
	rmmovq rA, D(rB)	
Execute	$valE \leftarrow valB + valC$	Efektif adresi hesapla
	popq rA	
Execute	$valE \leftarrow valB + 8$	Yığın pointer'ını arttır
	jXX Dest	
Execute		İşlem yok
	call Dest	
Execute	$valE \leftarrow valB + -8$	Yığın pointer'ını azalt
	ret	
Execute	$valE \leftarrow valB + 8$	Yığın pointer'ını arttır

```
int aluA = [
    icode in { IRRMOVQ, IOPQ } : valA;
    icode in { IIRMOVQ, IRMMOVQ, IMRMVQ } : valC;
    icode in { ICALL, IPUSHQ } : -8;
    icode in { IRET, IPOPOPQ } : 8;
    # Diğer komutlar ALU'ya gerek duymaz
```

ALU İşlemi

	OPI rA, rB
Execute	$valE \leftarrow valB \text{ OP } valA$
	cmovXX rA, rB
Execute	$valE \leftarrow 0 + valA$
	rmmovl rA, D(rB)
Execute	$valE \leftarrow valB + valC$
	popq rA
Execute	$valE \leftarrow valB + 8$
	jXX Dest
Execute	
	call Dest
Execute	$valE \leftarrow valB + -8$
	ret
Execute	$valE \leftarrow valB + 8$

ALU işlemini gerçekleştir

valA'yı ALU üzerinden aktar

Efektif adresi hesapla

Yığın pointer'ını arttır

İşlem yok

Yığın pointer'ını azalt

Yığın pointer'ını arttır

```
int alufun = [
    icode == IOPQ : ifun;
    1 : ALUADD;
];
```

```
bool set_cc =
    icode in {IOPQ};
```

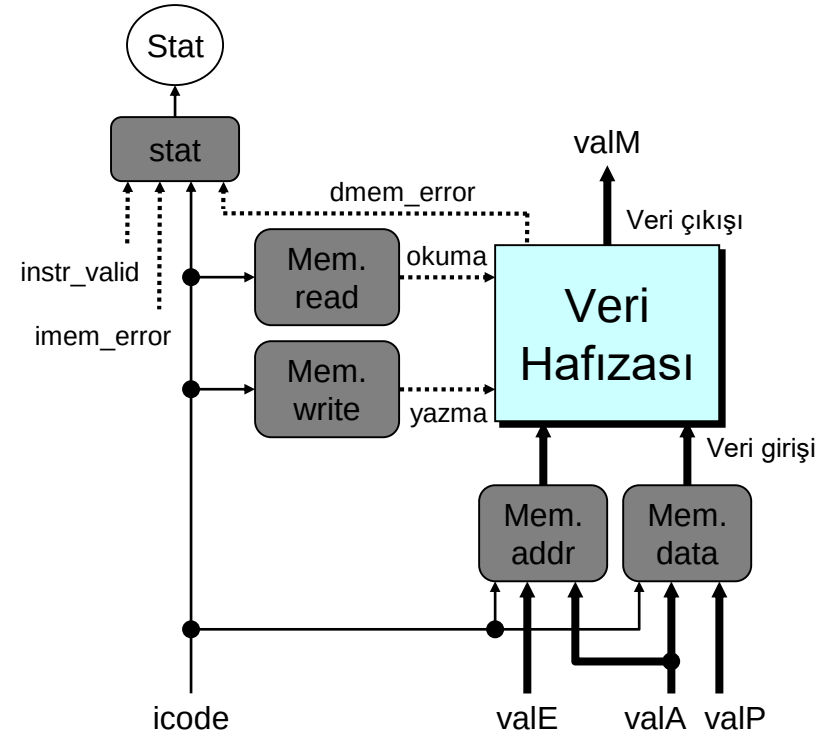
Memory Lojiği

Memory

- Hafızadan sözcük okunur veya hafızaya sözcük yazılır

Kontrol Lojiği

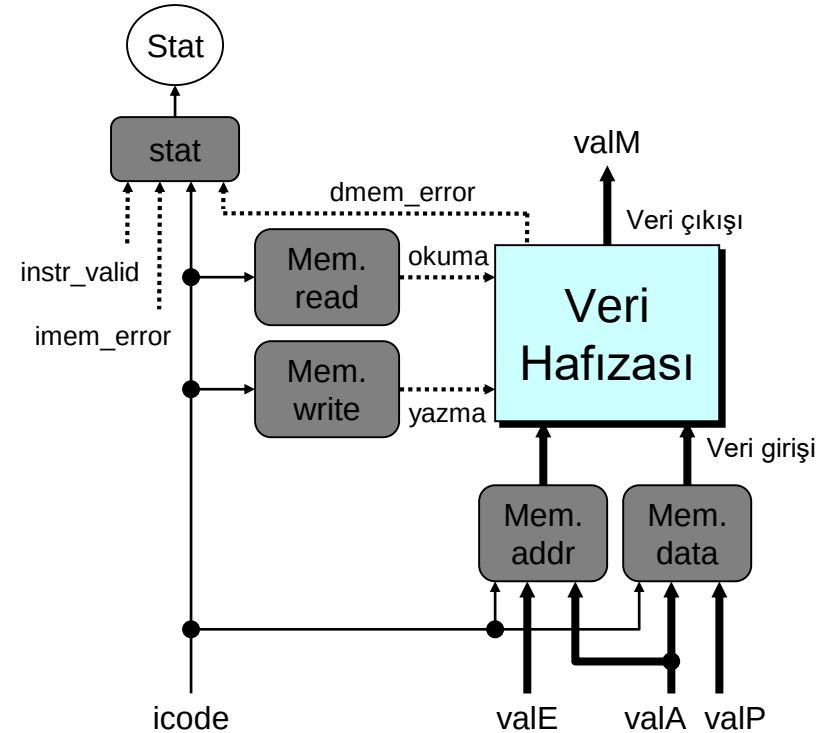
- stat: Komutun durumu nedir?
- Mem. read: sözcük okunacak mı?
- Mem. write: sözcük yazılacak mı?
- Mem. addr.: Adresi seçer
- Mem. data.: Veriyi seçer



Komut durumu

Kontrol Lojiği

- **stat**: Komut durumu nedir?



```
## Komut durumunu belirle
int Stat = [
    imem_error || dmem_error : SADR;
    !instr_valid: SINS;
    icode == IHALT : SHLT;
    1 : SAOK;
];
```

Memory Adresi

	OPq rA, rB	
Memory		İşlem yok
	rmmovq rA, D(rB)	
Memory	$M_8[\text{valE}] \leftarrow \text{valA}$	Değeri hafızaya yaz
	popq rA	
Memory	$\text{valM} \leftarrow M_8[\text{valA}]$	Yığından oku
	jXX Dest	
Memory		İşlem yok
	call Dest	
Memory	$M_8[\text{valE}] \leftarrow \text{valP}$	Dönüş adresini yığına yaz
	ret	
Memory	$\text{valM} \leftarrow M_8[\text{valA}]$	Dönüş adresini oku

```

int mem_addr = [
    icode in { IRMMOVQ, IPUSHQ, ICALL, IMRMVQ } : valE;
    icode in { IPOPOPQ, IRET } : valA;
    # Diğer komutlar adrese ihtiyaç duymaz

```

Memory Read (Okuma)

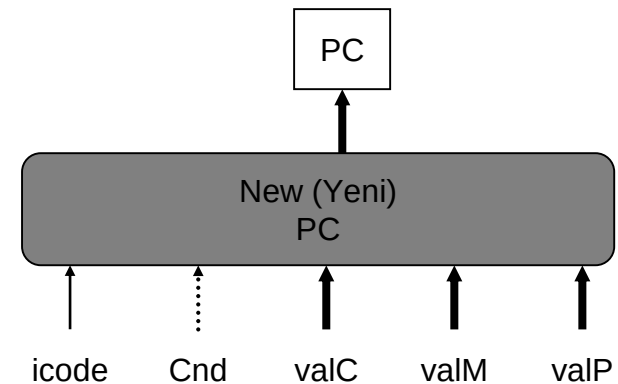
	OPq rA, rB	
Memory		İşlem yok
	rmmovq rA, D(rB)	
Memory	$M_8[valE] \leftarrow valA$	Değeri hafızaya yaz
	popq rA	
Memory	$valM \leftarrow M_8[valA]$	Yığından oku
	jXX Dest	
Memory		İşlem yok
	call Dest	
Memory	$M_8[valE] \leftarrow valP$	Dönüş adresini yığına yaz
	ret	
Memory	$valM \leftarrow M_8[valA]$	Dönüş adresini oku

```
bool mem_read = icode in { IMRMOVQ, IPOPOPQ, IRET };
```

PC Update (Güncelleme) Lojiği

New (Yeni) PC

- PC'nin bir sonraki değerini seç

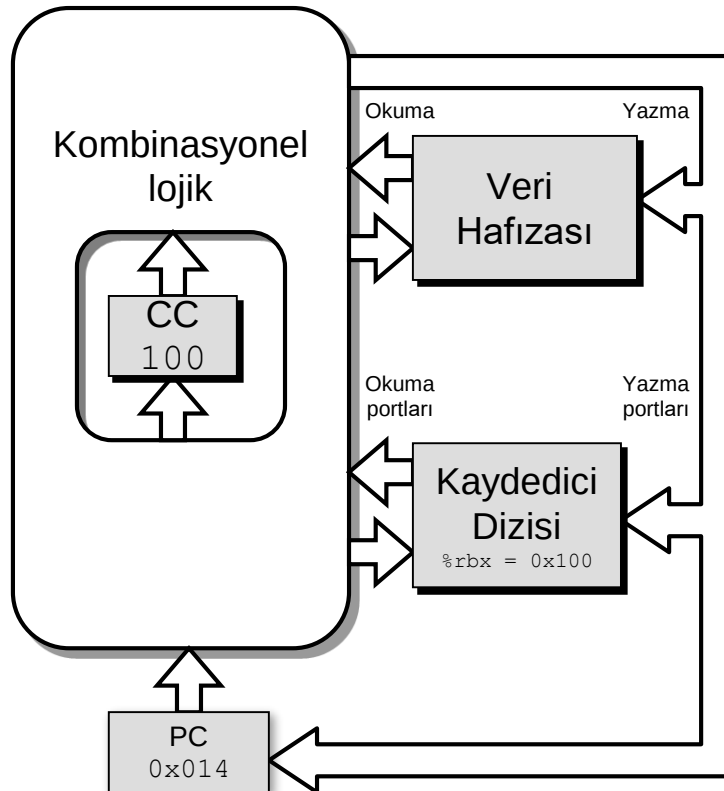


PC Update

	OPq rA, rB	
PC update	PC $\leftarrow$ valP	PC'yi güncelle
	rmmovq rA, D(rB)	
PC update	PC $\leftarrow$ valP	PC'yi güncelle
	popq rA	
PC update	PC $\leftarrow$ valP	PC'yi güncelle
	jXX Hedef	
PC update	PC $\leftarrow$ Cnd ? valC : valP	PC'yi güncelle
	call Hedef	
PC update	PC $\leftarrow$ valC	PC'ye Hedef adresini yükle
	ret	
PC update	PC $\leftarrow$ valM	PC'ye dönüş adresini yükle

```
int new_pc = [
    icode == ICALL : valC;
    icode == IJXX && Cnd : valC;
    icode == IRET : valM;
    1 : valP;
];
```

SEQ'in Çalışması



Durum

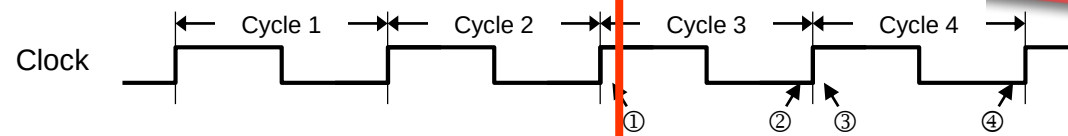
- PC kaydedicisi
- Durum kodu kaydedicisi
- Veri hafızası
- Kaydedici dizisi

Tamamı saat sinyalinin yükselen kenarında güncellenir

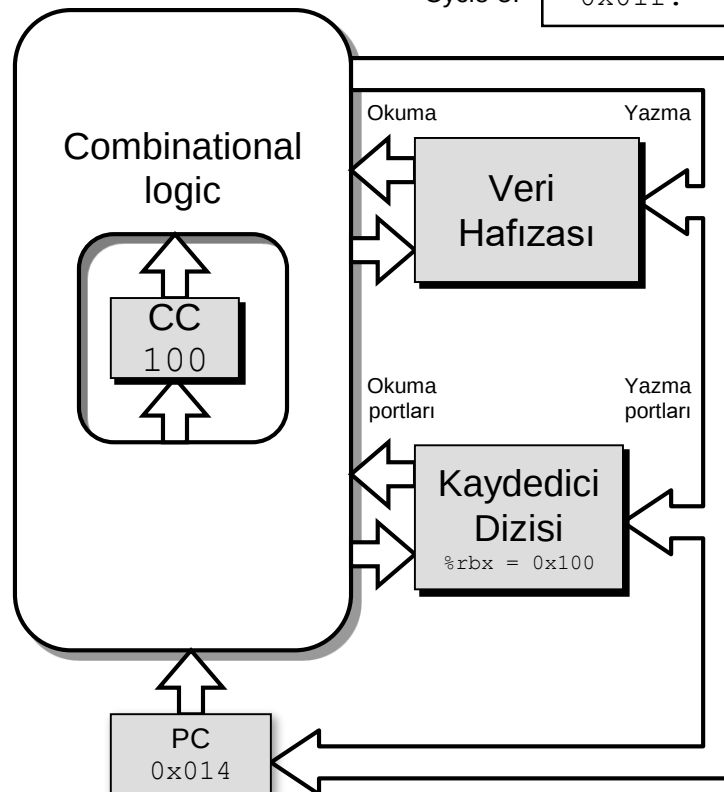
Kombinasyonel Lojik

- ALU
- Kontrol lojiği
- Hafıza okumaları
 - Komut hafızası
 - Kaydedici dizisi
 - Veri hafızası

SEQ'in Çalışması #2

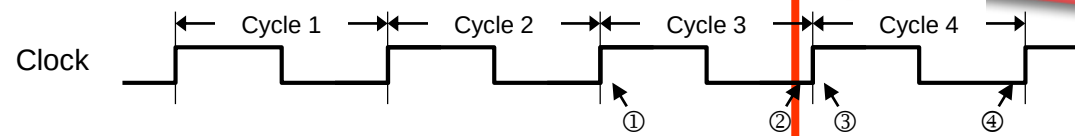


Cycle 1:	0x000:	irmovq \$0x100,%rbx	# %rbx <-- 0x100
Cycle 2:	0x00a:	irmovq \$0x200,%rdx	# %rdx <-- 0x200
Cycle 3:	0x014:	addq %rdx,%rbx	# %rbx <-- 0x300 CC <-- 000
Cycle 4:	0x016:	je hedef	# Dallanma olmaz
Cycle 5:	0x01f:	rmmovq %rbx,0(%rdx)	# M[0x200] <-- 0x300

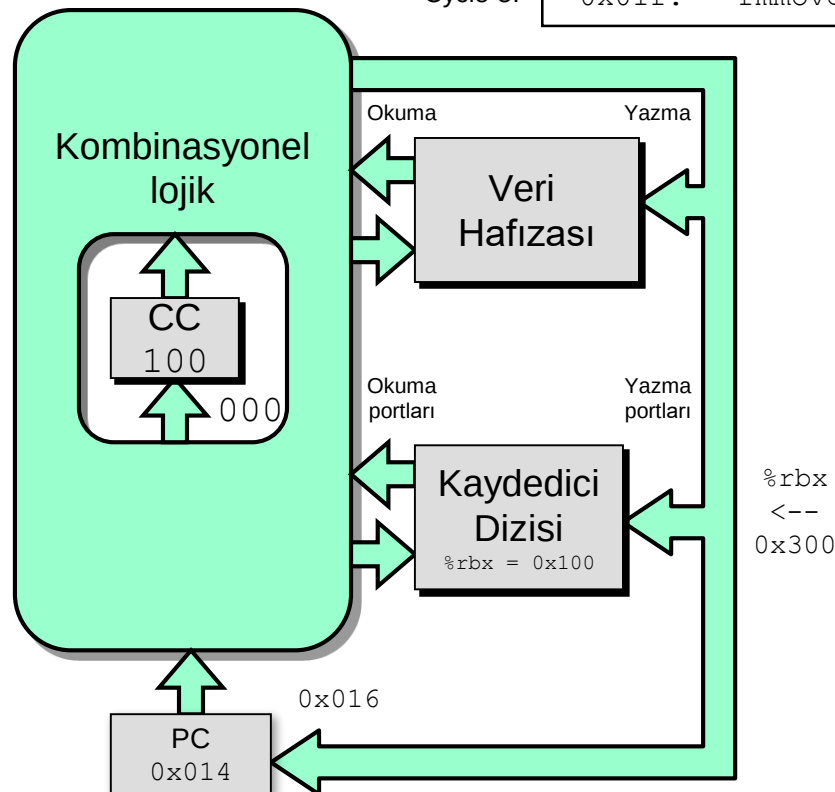


- durum ikinci `irmovq` komutuna bağlı olarak set edilir
- kombinasyonel lojik durum değişikliğine tepki vermeye başlar

SEQ'in Çalışması #3

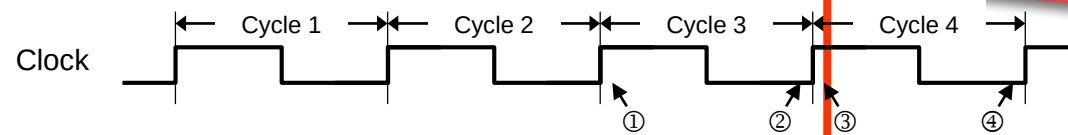


Cycle 1:	0x000:	irmovq \$0x100,%rbx	# %rbx <-- 0x100
Cycle 2:	0x00a:	irmovq \$0x200,%rdx	# %rdx <-- 0x200
Cycle 3:	0x014:	addq %rdx,%rbx	# %rbx <-- 0x300 CC <-- 000
Cycle 4:	0x016:	je hedef	# Dallanma olmaz
Cycle 5:	0x01f:	rmmovq %rbx,0(%rdx)	# M[0x200] <-- 0x300

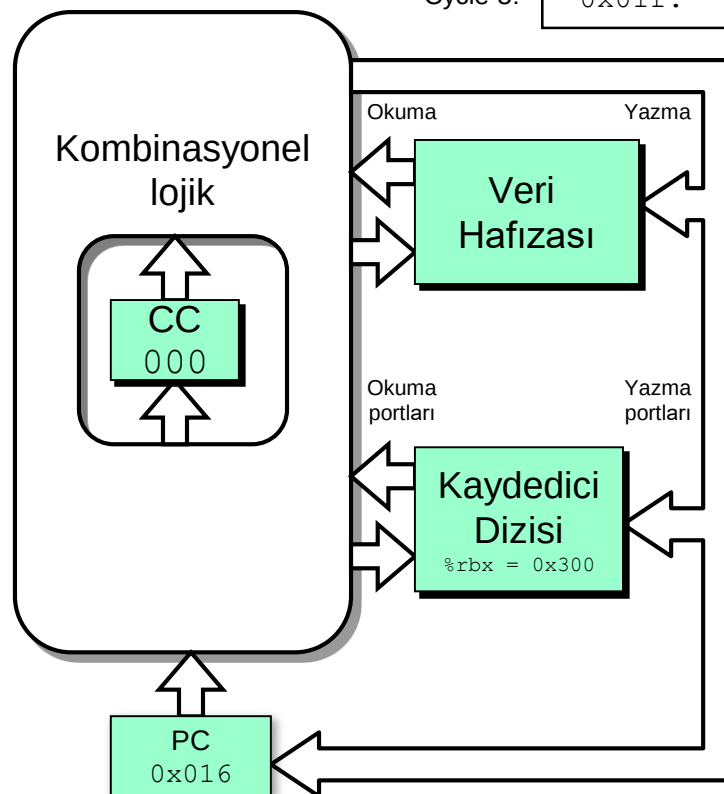


- durum ikinci `irmovq` komutuna bağlı olarak set edilir
- kombinasyonel lojik `addq` komutunun sonuçlarını oluşturur

SEQ'in Çalışması #4

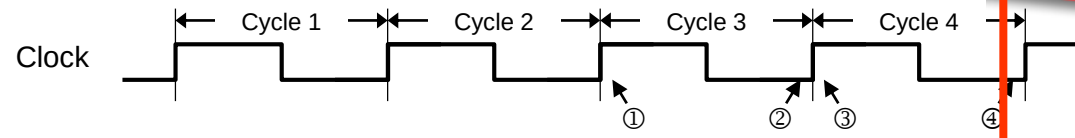


Cycle 1:	0x000:	irmovq \$0x100,%rbx	# %rbx <-- 0x100
Cycle 2:	0x00a:	irmovq \$0x200,%rdx	# %rdx <-- 0x200
Cycle 3:	0x014:	addq %rdx,%rbx	# %rbx <-- 0x300 CC <-- 000
Cycle 4:	0x016:	je hedef	# Dallanma olmaz
Cycle 5:	0x01f:	rmmovq %rbx,0(%rdx)	# M[0x200] <-- 0x300

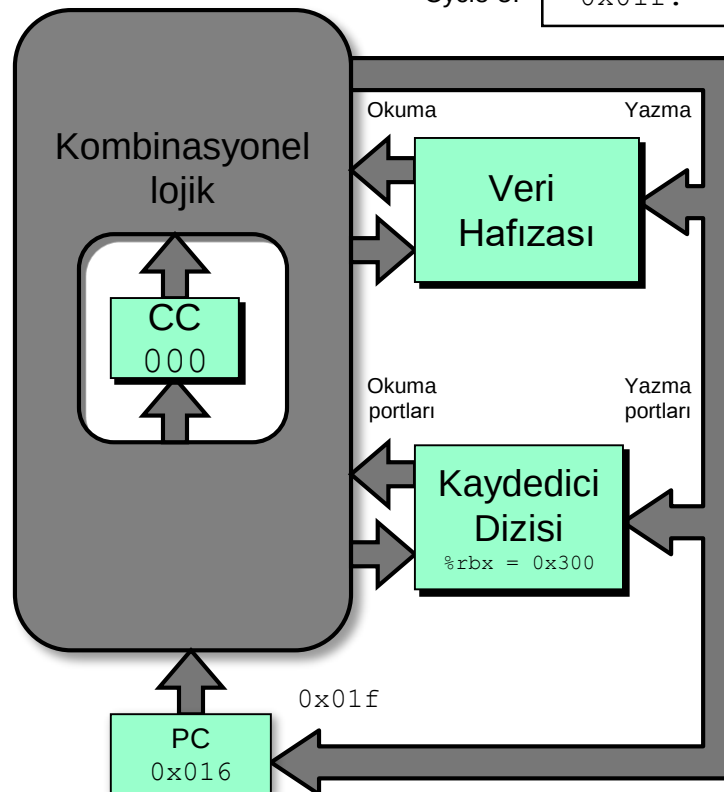


- durum addq komutuna bağlı olarak set edilir
- kombinasyonel lojik durum değişikliğine tepki vermeye başlar

SEQ'in Çalışması #5



Cycle 1:	0x000:	irmovq \$0x100,%rbx	# %rbx <-- 0x100
Cycle 2:	0x00a:	irmovq \$0x200,%rdx	# %rdx <-- 0x200
Cycle 3:	0x014:	addq %rdx,%rbx	# %rbx <-- 0x300 CC <-- 000
Cycle 4:	0x016:	je hedef	# Dallanma olmaz
Cycle 5:	0x01f:	rmmovq %rbx,0(%rdx)	# M[0x200] <-- 0x300



- durum addq komutuna bağlı olarak set edilir
- kombinasyonel lojik je komutunun sonuçlarını oluşturur

SEQ Özeti

Uygulaması

- Herbir komut basit adımlar dizisi olarak ifade edilir
- Her komut tipi için aynı genel akış izlenir
- Kaydedici, hafızalar ve öntasarımlı kombinasyonel blokları birleştirir
- Bu blokların birleşimini kontrol lojiği ile sağlar

Kısıtlamalar

- Pratik uygulama açısından çok yavaştır
- Bir saat palsinde, komut hafızası, kaydedici dizisi, ALU ve veri hafızası boyunca sinyallerin yayılmaları gerekir
- Bu yayılma gereksinimi nedeniyle saat sinyalinin çok yavaş işlemesi gerekir
- Donanım alt birimleri saat palsinin sadece belli bir kısmında aktif olarak çalışırlar (saat palsinin geri kalan süresinde sadece beklerler, herhangi bir işlem gerçekleştirmezler)

Kaynaklar

- 1. Computer Systems: A Programmer's Perspective, Randal E. Bryant ve David R. O'Hallaron, PEARSON.**

SÜLEYMAN DEMİREL ÜNİVERSİTESİ



TEŞEKKÜRLER